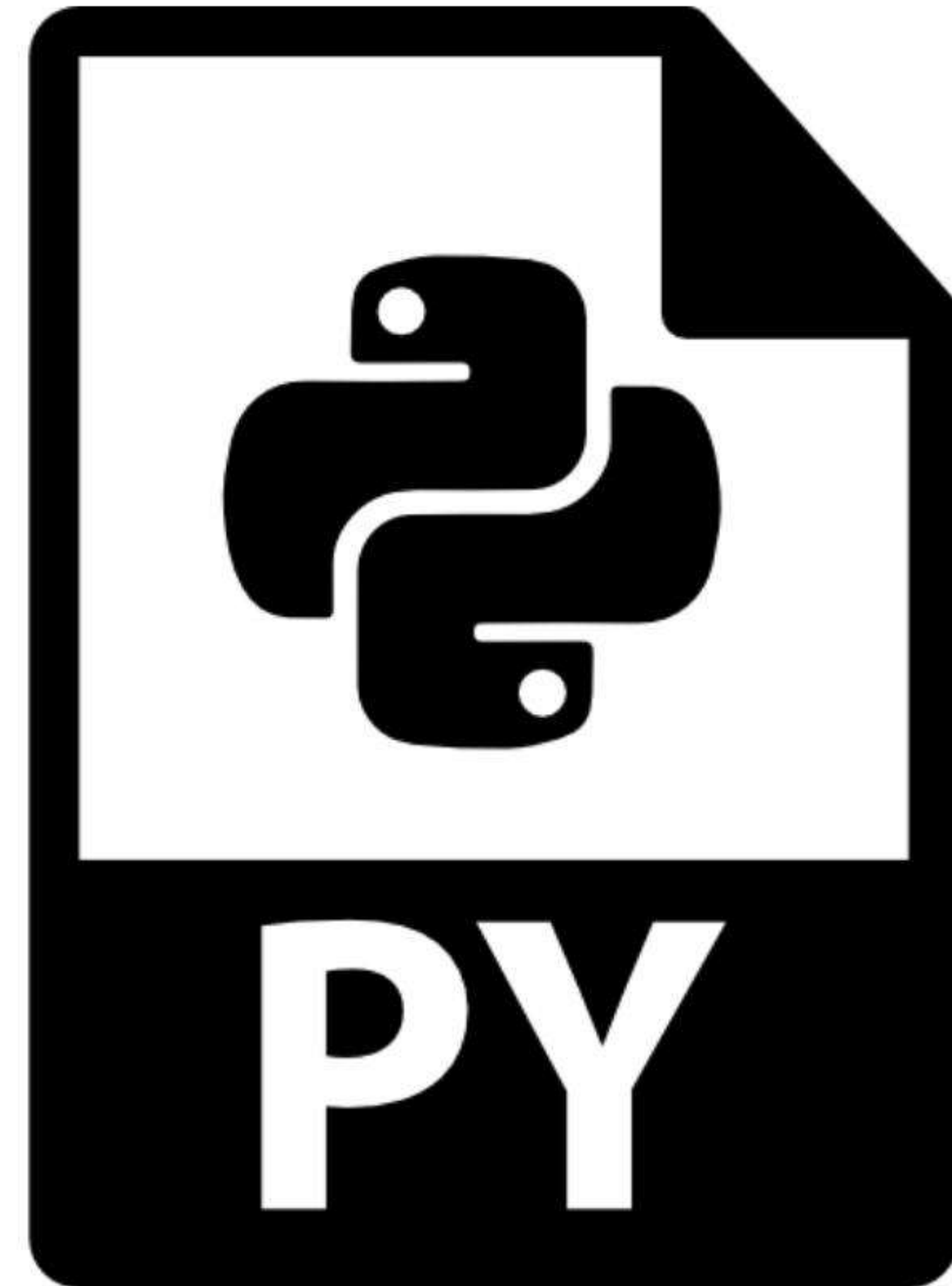


PYTHON TUTORIAL

Sum of Two Files

Combining data from multiple sources with arithmetic operations and precise formatting.

Prepared for Python Learners

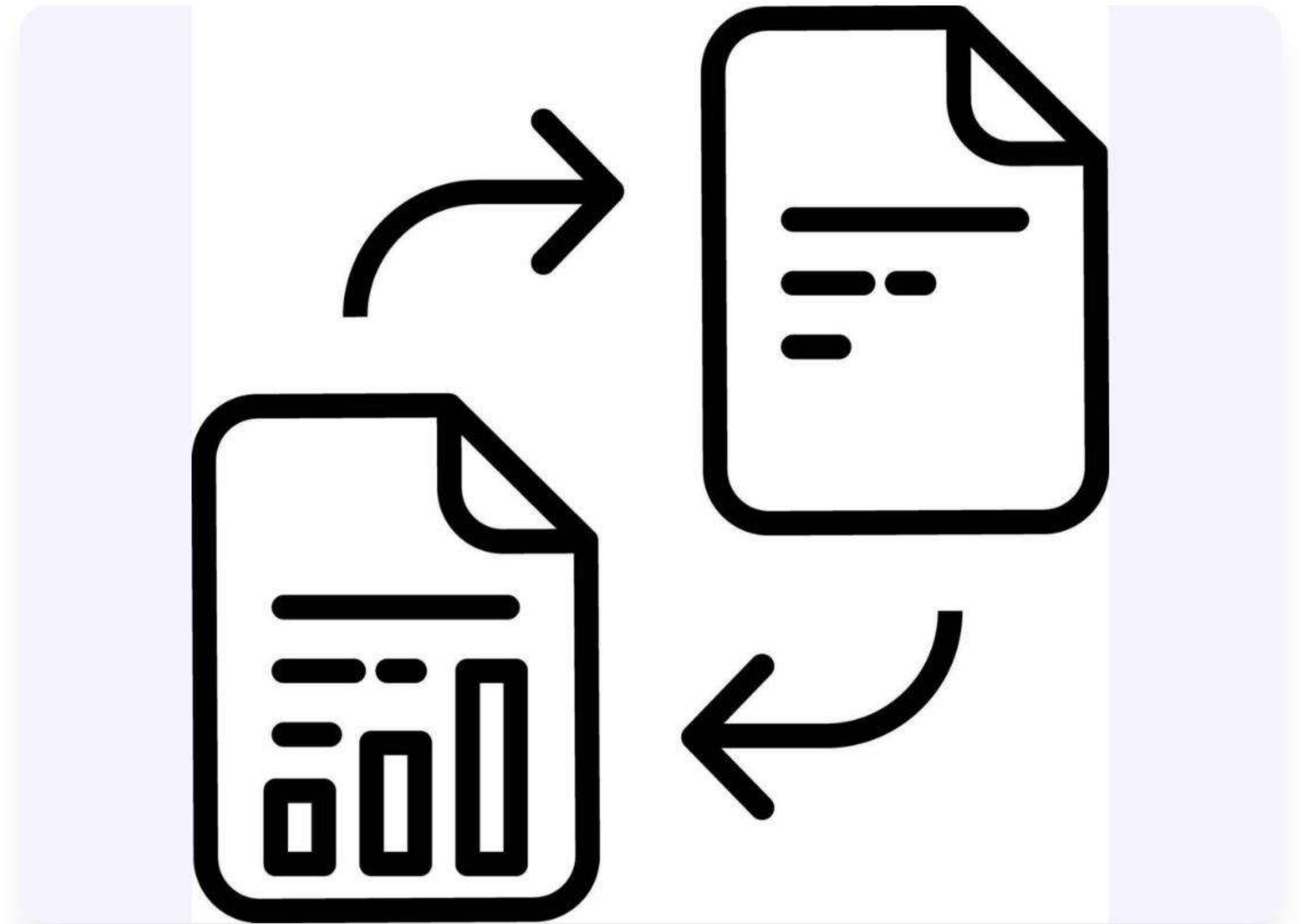


The Challenge

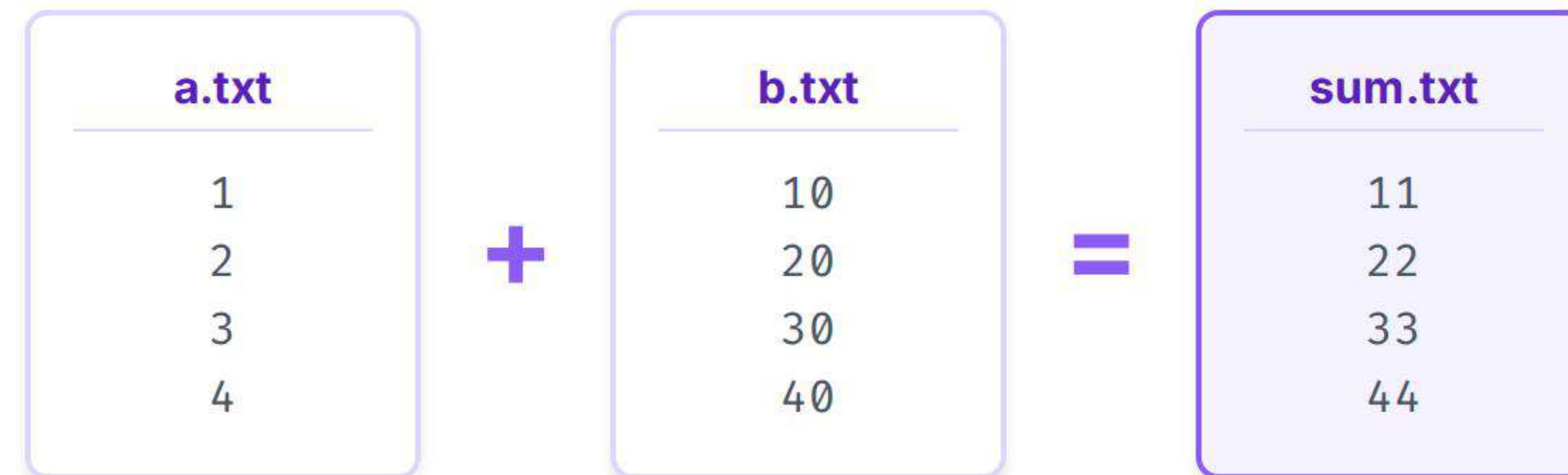
Goal

We have two files, `a.txt` and `b.txt`, both containing integers. We need to create `sum.txt`.

- ✓ **Line-by-Line Sum:** Line 1 of *a* + Line 1 of *b* = Line 1 of *sum*.
- ✓ **Type Conversion:** Inputs are strings; math requires integers.
- ✓ **Strict Formatting:** No extra blank lines at the end of the file.



Visualizing the Process



Note: `str("1") + str("10") = "110"` (Wrong)

`int("1") + int("10") = 11` (Correct)

Key Concept: The `zip()` Function

Parallel Iteration

The `zip()` function takes two or more iterables (like lists or files) and aggregates them into a tuple.

It's like a zipper on a jacket: it brings teeth from the left and right sides together, one by one.

```
zip([1, 2], [10, 20])  
→ (1, 10), (2, 20)
```



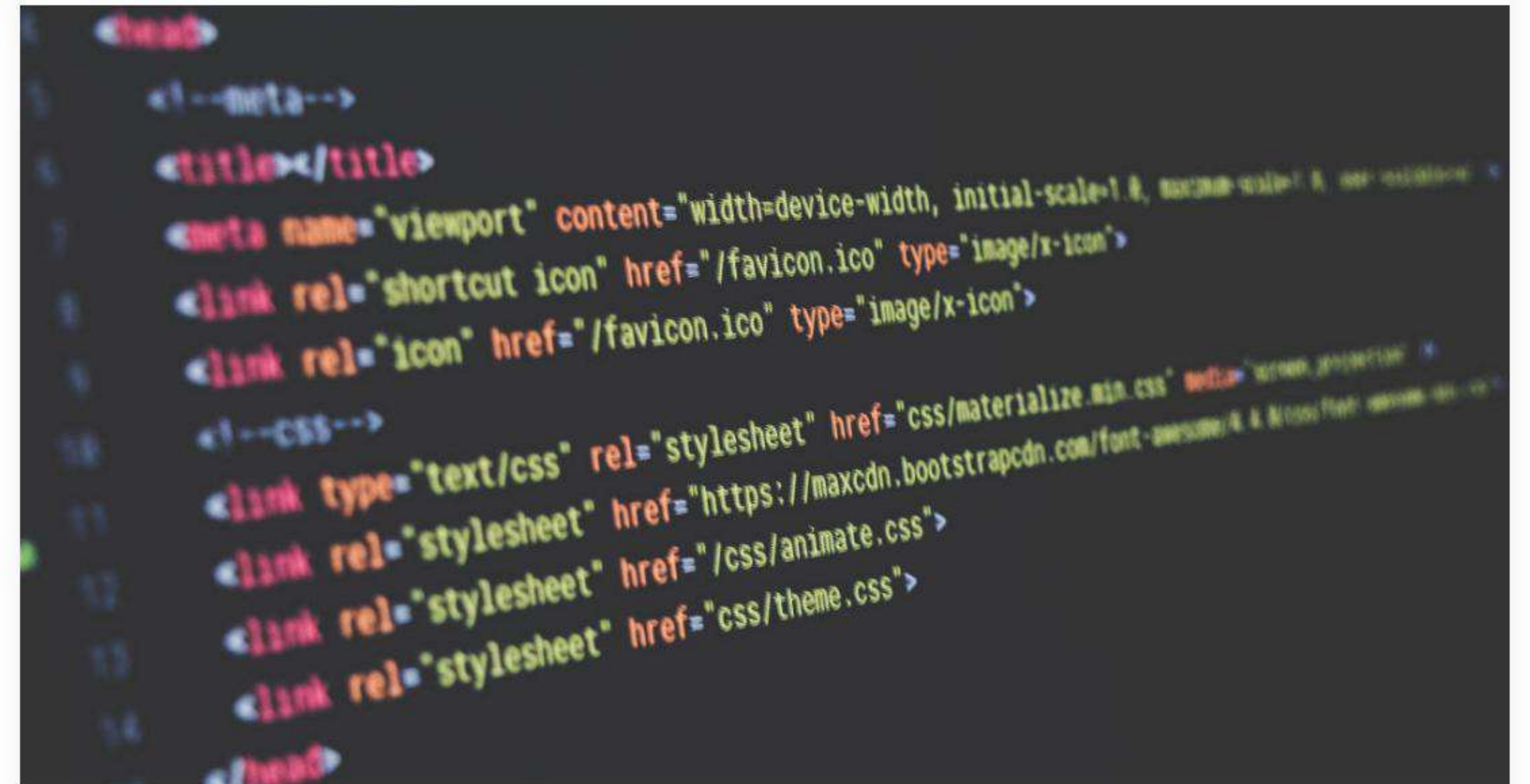
Solution 1: Iterative Approach

Line-by-Line Processing

We open all three files simultaneously. Using a loop with `zip()`, we grab one line from *a* and one line from *b* at the same time.

Pros

- ✓ **Memory Efficient:** Ideal for massive files (e.g., millions of lines) because we only hold two numbers in memory at once.
- ✓ **Standard Pattern:** Easy to understand logic.



```
<head>
  <!--meta-->
  <title></title>
  <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no">
  <link rel="shortcut icon" href="/favicon.ico" type="image/x-icon">
  <link rel="icon" href="/favicon.ico" type="image/x-icon">
  <!--CSS-->
  <link type="text/css" rel="stylesheet" href="css/materialize.min.css" media="screen, projection">
  <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/font-awesome/4.4.0/css/font-awesome.min.css">
  <link rel="stylesheet" href="/css/animate.css">
  <link rel="stylesheet" href="css/theme.css">
</head>
```


Flowchart: Iterative Approach



Open Files

Open A, B (read)
& Sum (write)



Zip Lines

Get next (lineA, lineB) pair



Calculate

`int(lineA) +`
`int(lineB)`



Write

Write sum + `\n`



Repeat

Until files end

Code: Iterative Solution

```
def sum_files_iterative():
    try:
        with open('a.txt', 'r') as f1, \
             open('b.txt', 'r') as f2, \
             open('sum.txt', 'w') as out:

            # zip() reads both files in parallel
            for line1, line2 in zip(f1, f2):
                try:
                    num1 = int(line1.strip())
                    num2 = int(line2.strip())
                    out.write(f"{num1 + num2}\n")
                except ValueError:
                    pass # Skip non-integer lines

    except FileNotFoundError:
        print("One of the files is missing.")
```

Code Breakdown

- `zip(f1, f2)`: Automatically stops when the shortest file ends.
- `int(line.strip())`: Removes whitespace/newlines and converts to integer.
- `write( ... \n)`: Adds a newline after every sum (Be careful: this adds a newline to the last line too).

Solution 2: Precise Formatting Approach

The "No Trailing Newline" Logic

The standard `write( ... \n)` loop always leaves a blank line at the end of the file. To solve the challenge strictly:

- ✓ **Step 1:** Calculate all sums and store them in a list.
- ✓ **Step 2:** Join the list into a single string using `\n` as a separator.
- ✓ **Result:** Newlines appear *between* numbers, but not after the last one.



1 —
2 — List Collection

Flowchart: Precise Formatting



1. Read & Zip

Read all lines from both files
and pair them up
immediately.



2. Compute List

Create a list of strings: ['11',
'22', '33', '44'].



3. Join & Write

`"\n".join(list)` creates one
string. Write it once.

Code: Precise Solution

```
def sum_files_precise():
    try:
        with open('a.txt') as f1, open('b.txt') as f2:
            # Create list of sum strings
            sums = [
                str(int(a) + int(b))
                for a, b in zip(f1, f2)
                if a.strip() and b.strip()
            ]

            with open('sum.txt', 'w') as out:
                # Join with newline separator
                out.write("\n".join(sums))

    except ValueError:
        print("Invalid data found")
```

Code Breakdown

- `str(int(a) + int(b))`: Calculates sum and converts back to string for joining.
- `"\n".join(sums)`: The magic trick. It inserts a newline between elements but **never** at the end.
- `if a.strip()`: Ensures we don't try to process empty lines.

Handling Edge Cases

1. Unequal File Lengths

What if `a.txt` has 5 lines and `b.txt` has 3?

- ✓ **Default `zip()`:** Stops at the shortest file (3 lines). Extra data is ignored.
- ✓ **`itertools.zip_longest()`:** Fills missing values with a default (e.g., 0). Use this if you want to sum *everything*.

Using `zip_longest`:

```
from itertools import zip_longest

# Uses 0 if a line is missing
for a, b in zip_longest(f1, f2, fillvalue="0"):
    total = int(a) + int(b)
```


Comparing Approaches

Feature	Iterative (Loop)	Precise (Join)
Formatting	Usually adds trailing newline	Perfect (No trailing newline)
Memory Usage	Low (Streaming)	Higher (Stores output in list)
Code Style	Procedural	Functional / Pythonic
Best Use Case	Large data processing	Strict format requirements